Capstone Project Report
Great Learning Wipro Pre-Skilling Training

PROJECT TITLE : ProfileBook

BY VENKATA SAI REHAN
SRM AP University.

Preface

This document presents the **Use Case Documentation and Required Specifications and a total report of what was done so far by me In the ProfileBook**, a web-based social networking platform designed to allow users to interact, communicate, and share content in an online environment.

The purpose of this document is to clearly describe the **functional behavior of the system**, the interactions between users and the platform, and the architectural design followed during the development of the application.

This document outlines the **project overview, development methodology, system users, use case analysis, database design, and system implementation details**. It provides a structured explanation of how the system is designed and how different components of the application work together to achieve the overall functionality of the platform.

The document begins with an overview of the ProfileBook project and the technologies used for its development. It then explains the **Agile Software Development Life Cycle** adopted for the project and describes how the system was developed in multiple stages. Following that, the document presents the **user stories**, which describe the system functionalities from the perspective of both users and administrators.

The document also explains the **different types of users interacting with the system**, along with their responsibilities and system privileges. In addition, the **Use Case Diagram** is presented to visually represent the interactions between users and system functionalities.

Furthermore, the document provides a detailed explanation of the **database design implemented using Entity Framework Core with the Code-First approach**, where C# model classes are used to generate the database schema in SQL Server.

Finally, the document explains the **backend architecture implemented using ASP.NET Core Web API** and the **frontend interface developed using Angular**, highlighting the structure of the application and the components implemented so far.

This documentation serves as a comprehensive reference for understanding the **design, functionality, and implementation approach of the ProfileBook system**.

Table Of Contents:

1).About the Project

2).Software Development Life Cycle (SDLC)

2.1 Types of SDLC Models

2.2 Agile Software Development Model

2.3 Design Principles Used

2.4 Design Patterns Used

2.5 Agile Implementation in ProfileBook

3).Types of Users

3.1 Normal User

3.2 Administrator (Admin)

4). User and Admin Stories

4.1 User Stories

4.2 Admin Stories

5).Figma Brainstorming

6).Use Case Diagram

7).Database Design

8).Backend Architecture

8.1Controllers

8.2 Models

8.3 Data (DbContext)

8.4 DTOs

8.5 Migrations

8.6 Design Patterns

8.7 Services and Repositories

8.8 Helpers and Authentication

8.9 SignalR Real-Time Communication

9).Frontend Implementation

10).Testing

11). Final Completion Status

12).Future Improvements

13).Conclusion

14).Output

Note: System Refers to Web API or Web Application.

1). About the Project

ProfileBook is a web-based social networking platform designed to enable users to communicate and interact with each other in a virtual environment. The system allows users to share content, connect with other users, and participate in various social interactions such as liking posts, commenting on posts, sending messages, and reporting inappropriate behavior.

The platform supports two primary roles within the system: **User** and **Administrator (Admin)**. Each role is designed with specific permissions and responsibilities to ensure smooth system operation and proper moderation of content.

A **User** is a registered member of the platform who can create posts, view posts shared by other users, like and comment on posts, send messages, search for other users, and report users who violate platform guidelines. Users can also manage their personal profile information within the system.

An **Administrator (Admin)** is responsible for managing the overall platform activities. The admin has elevated privileges that allow them to monitor system operations and ensure that the platform is used appropriately. Administrative responsibilities include approving or rejecting posts submitted by users, managing user accounts, reviewing reported users, and creating or managing user groups.

The system follows a **client-server architecture**, where the frontend user interface is developed using **Angular**, while the backend services are implemented using **ASP.NET Core Web API**. Communication between the frontend and backend is handled through RESTful APIs.

The application uses **SQL Server** as the database management system to store application data such as users, posts, messages, reports, and groups. **Entity Framework Core (EF Core)** is used as the Object Relational Mapping (ORM) tool to simplify database interactions and manage data efficiently through C# model classes.

The primary objective of ProfileBook is to provide a structured and secure social networking platform that enables user interaction while maintaining administrative control over system activities and content moderation.

2). Software Development Life Cycle (SDLC)

The **Software Development Life Cycle (SDLC)** is a systematic process used for developing software applications in a structured and organized manner. It provides a framework that guides the development team through different phases of software creation, ensuring that the final system meets the required quality standards and functional requirements.

The SDLC process helps developers plan, design, develop, test, and maintain software efficiently. By following a defined life cycle, development teams can manage project complexity, reduce risks, and improve the overall quality of the software product.

Several SDLC models are used in software development depending on the nature of the project and its requirements.

2.1 Types of SDLC Models

Different software development models are used to structure the development process. Some of the commonly used SDLC models include:

- Waterfall Model
- Agile Model
- Spiral Model
- V-Model
- Incremental Model
- Iterative Model

Each model provides a different approach for organizing development activities.

In this project, we will not discuss all SDLC models in detail. Instead, we focus on the **Agile Software Development Model**, which has been adopted for the development of the ProfileBook system.

2.2 Agile Software Development Model

The **Agile Software Development Model** is an iterative and incremental development methodology that focuses on delivering software in small functional units known as **Sprints**.

Unlike traditional development approaches where the entire system is developed in a single phase, Agile divides the project into smaller development cycles. Each sprint focuses on implementing and delivering specific functionalities of the system.

Agile promotes continuous collaboration, frequent feedback, and flexibility in adapting to changing requirements. This approach allows developers to build and test system features gradually while ensuring that the system evolves according to user needs.

Because of its flexibility and efficiency, Agile is widely used for modern web application development.

2.3 Design Principles Used in the System

During the development of the ProfileBook system, several software design principles were followed to ensure that the system architecture remains modular, maintainable, and scalable.

The backend system follows concepts inspired by the **SOLID design principles**, which help developers create clean and flexible code.

One of the important principles applied is the **Single Responsibility Principle (SRP)**. According to this principle, each class should have only one responsibility. In this project, controllers are responsible for handling HTTP requests, while services and repositories manage business logic and data access operations.

Another important concept applied is **Dependency Injection**, which is supported by ASP.NET Core. Dependency injection helps reduce tight coupling between system components and makes the application easier to maintain and test.

DTOs (Data Transfer Objects) are also used to transfer data between the client and server while protecting internal model structures.

2.4 Design Patterns Used

To improve the overall architecture of the system, several design patterns were implemented.

Factory Pattern

The **Factory Design Pattern** is used to create objects without exposing the creation logic to the client. In this project, the **NotificationFactory** class is responsible for generating different types of notifications based on system events.

This pattern helps in managing object creation in a flexible and extensible way.

Observer Pattern

The **Observer Pattern** is implemented to handle notification updates and event-based communication within the system.

The following components are used:

- **INotificationObserver** – Defines the observer interface.
- **NotificationSubject** – Maintains and manages observers.
- **NotificationService** – Handles the delivery of notifications.

This pattern enables the system to notify subscribed components when certain events occur.

State Pattern

The **State Pattern** is used to manage different states of system entities. In this project, it can be used to represent different states of posts such as **Pending**, **Approved**, and **Rejected**.

This approach simplifies state management and avoids complex conditional statements in the system.

2.5 Agile Implementation in ProfileBook

The development of the ProfileBook system follows the Agile methodology, where the project was divided into multiple development phases called **Sprints**.

Sprint Planning and Implementation

Although the project specification already defined the development process into major sprints, I further organized the work into manageable stages to complete the tasks efficiently. Since the development of the ProfileBook system was carried out individually, I divided the work into smaller phases within the sprints to ensure steady progress and faster achievement of the project objectives.

By structuring the work in this way, I was able to focus on specific components of the system in each phase, perform testing during development, and gradually integrate all parts of the application.

The implementation of the ProfileBook system was mainly carried out in the following three development sprints.

Sprint 1 – Database Design and Backend Setup

During the first sprint, I focused on designing the **database structure and setting up the backend foundation** of the system. I created the core system entities such as **Users, Posts, Messages, Reports, and Groups** using C# model classes in ASP.NET Core.

Using **Entity Framework Core with the Code-First approach**, I generated the corresponding database tables in SQL Server from the model classes. After creating the initial database structure, I refined the schema by defining **primary keys, foreign keys, and nullable or non-nullable constraints** to maintain proper data integrity.

For security reasons, I created the **Admin account manually using an SQL query**, ensuring that administrative privileges cannot be created through the user registration process.

During this phase, I also implemented the **UserController**, which handles user-related operations such as user registration, login authentication, and retrieving user information from the database.

Sprint 2 – Backend API Development and Frontend Setup

In the second sprint, I focused on developing the **backend APIs and setting up the frontend application**. I implemented several backend controllers using **ASP.NET Core Web API** to manage different system functionalities.

Controllers such as **UserController, PostController, MessageController, and ReportController** were created to handle requests from the frontend and interact with the database through Entity Framework Core.

At the same time, I initialized the frontend application using **Angular** and developed the initial user interface pages, including the **Welcome Page, Login Page, and User Registration Page**.

Sprint 3 – Feature Integration and Admin Dashboard

During the third sprint, I focused on integrating the frontend components with the backend APIs and implementing administrative functionalities.

I developed the **Admin Dashboard** to allow administrators to manage users, review reported users, and monitor system activities. Additional system features such as post interactions, messaging functionality, and further improvements are being integrated to complete the system.

2.6 Testing

During the development of the ProfileBook system, testing was performed continuously to ensure that the implemented features function correctly and that the system components interact properly. Basic verification of the system was conducted after implementing each feature to confirm that the application behaves as expected.

Initial **Smoke Testing** was performed to verify that the core backend APIs were functioning correctly. The APIs such as **user registration and user login** were tested using the **Swagger interface** provided by ASP.NET Core. These tests ensured that the backend services were able to receive requests, process them correctly, and return appropriate responses.

After confirming the API functionality, **Functional Testing** was carried out to verify that the system features operate according to the specified requirements. This included testing the **user registration process, login functionality, and admin access to the dashboard** through the Angular frontend.

In addition, **Integration Testing** was performed to ensure proper communication between the **Angular frontend, ASP.NET Core Web API backend, and SQL Server database**. These tests verified that user input from the frontend is correctly processed by the backend APIs and stored or retrieved from the database as expected.

Basic **UI Testing** was also conducted to ensure that the user interface components such as the **Welcome Page, Login Page, Signup Page, and Admin Dashboard** are displayed correctly and provide a smooth user interaction experience.

All testing screenshots and related materials are maintained in the **GitHub repository of the project inside Project_information**, where the implemented features and their testing results are documented.

GitHub Repository Link:
https://github.com/GVSRehan/ProfileBook_Wipro_Training_Venkata_Sai_Rehan

3).Types of Users

The ProfileBook system supports multiple types of users who interact with the platform based on their roles and permissions. Each user type has specific responsibilities and access privileges that define how they interact with the system.

In the ProfileBook application, two primary types of users are identified: **Normal Users** and **Administrators (Admin)**. These users interact with the system in different ways depending on their roles and permissions.

3.1 Normal User

A **Normal User** is a registered member of the ProfileBook platform who uses the Web site to interact with other users and participate in social networking activities.

Users can create accounts using the registration process and log in to the web site using their credentials. Once authenticated, users can perform several actions such as viewing posts shared by other users, creating new posts, liking and commenting on posts, sending messages to other users, and reporting inappropriate behavior if necessary.

Users are also able to manage their personal profile information within the system. The system ensures that user-generated content follows platform guidelines by sending newly created posts for administrative approval before they become visible to other users.

The primary goal of the normal user role is to enable interaction and communication within the platform while maintaining a structured and moderated environment.

3.2 Administrator (Admin)

An **Administrator (Admin)** is a privileged user responsible for managing the system and ensuring that the platform operates smoothly and securely.

Administrators have elevated permissions compared to normal users and are responsible for monitoring system activities and enforcing platform rules. One of the key responsibilities of the administrator is reviewing and approving posts submitted by users before they become publicly visible.

In addition to post moderation, administrators are responsible for managing user accounts, reviewing reports submitted by users, and taking appropriate action when violations occur. Administrators can also create and manage user groups to organize users within the system.

For security reasons, administrative accounts are not created through the normal registration process. Instead, they are inserted directly into the database using controlled administrative access to prevent unauthorized users from obtaining administrative privileges.

The administrator role plays an important part in maintaining the reliability, safety, and proper functioning of the ProfileBook system.

4).User and Admin Stories

User stories describe the system requirements from the perspective of the people who interact with the application. They help explain what functionality the system should provide and how different users expect the system to behave.

In the ProfileBook system, user stories are defined for both **Normal Users** and **Administrators** to represent the actions they can perform within the platform.

4.1 User Stories

The following user stories describe the expected functionalities available to normal users of the system.

- As a **User**, I should be able to **register and create an account** in the system.
- As a **User**, I should be able to **log into the system using my credentials**.
- As a **User**, I should be able to **view posts shared by other users**.
- As a **User**, I should be able to **create posts and submit them for admin approval**.
- As a **User**, I should be able to **like and comment on posts shared by other users**.
- As a **User**, I should be able to **send messages to other users**.
- As a **User**, I should be able to **search for other users in the platform**.
- As a **User**, I should be able to **report inappropriate users to the administrator**.

4.2 Admin Stories

The following user stories describe the responsibilities and actions available to the administrator.

- As an **Admin**, I should be able to **log into the system**.
- As an **Admin**, I should be able to **view and manage all users in the system**.

- As an **Admin**, I should be able to **approve or reject posts submitted by users**.
- As an **Admin**, I should be able to **view users who have been reported by other users**.
- As an **Admin**, I should be able to **create and manage user groups**.

5).Figma Brainstorming

Before starting the implementation of the frontend interface, I used **Figma** as a brainstorming and planning tool to visualize the structure and workflow of the ProfileBook web application. This step helped me to organize the system components and understand how different modules of the application interact with each other.

During this brainstorming phase, I mapped out the **major actors of the system (User and Admin)** along with the main functionalities such as **user authentication, post management, messaging, reporting, and administrative operations**. The diagram also helped in identifying the relationships between different modules and how they interact with the database.

This planning stage allowed me to clearly define the system flow before beginning the actual development of the backend APIs and frontend components. By organizing the features visually, I was able to plan the architecture of the system more effectively and ensure that the implementation followed a structured approach.

The following figure represents the brainstorming diagram created in Figma, which outlines the high-level structure of the ProfileBook system.

Figure 5.1: ProfileBook System Brainstorming Diagram (Figma)

Figma Link
https://www.figma.com/board/Ep58i50BZDZxA65sFx39Cr/Untitled?node-id=0-1&p=f&t=zmUenVerygir4lEz-0

6).Use Case Diagram

A **Use Case Diagram** is used to represent the functional requirements of a system by illustrating the interaction between users and the system. It provides a high-level view of how different actors interact with the system and what operations they can perform.

In the ProfileBook web application, the use case diagram helps in identifying the main functionalities available to the two primary actors of the system: **User** and **Administrator (Admin)**.

The **User** interacts with the platform through the website interface and performs activities such as registering an account, logging into the system, viewing posts, creating posts, sending messages, liking or commenting on posts, searching for other users, and reporting inappropriate users.

The **Administrator (Admin)** is responsible for managing the platform and maintaining system control. Administrative functionalities include logging into the admin system, managing users, reviewing and approving posts submitted by users, viewing reported users, and managing user groups.

The use case diagram provides a clear visual representation of these interactions and helps in understanding the responsibilities of each actor within the system.

Note:

To see All the Use cases Diagram Please Visit my Github Repo Link In that Find Use Case Document there

https://github.com/GVSRehan/ProfileBook_Wipro_Training_Venkata_Sai_Rehan
If I keep All the use Cases Documents in this report it doesn't look good so please head my repo

Use Cases Diagram U can find in UseCase Folder

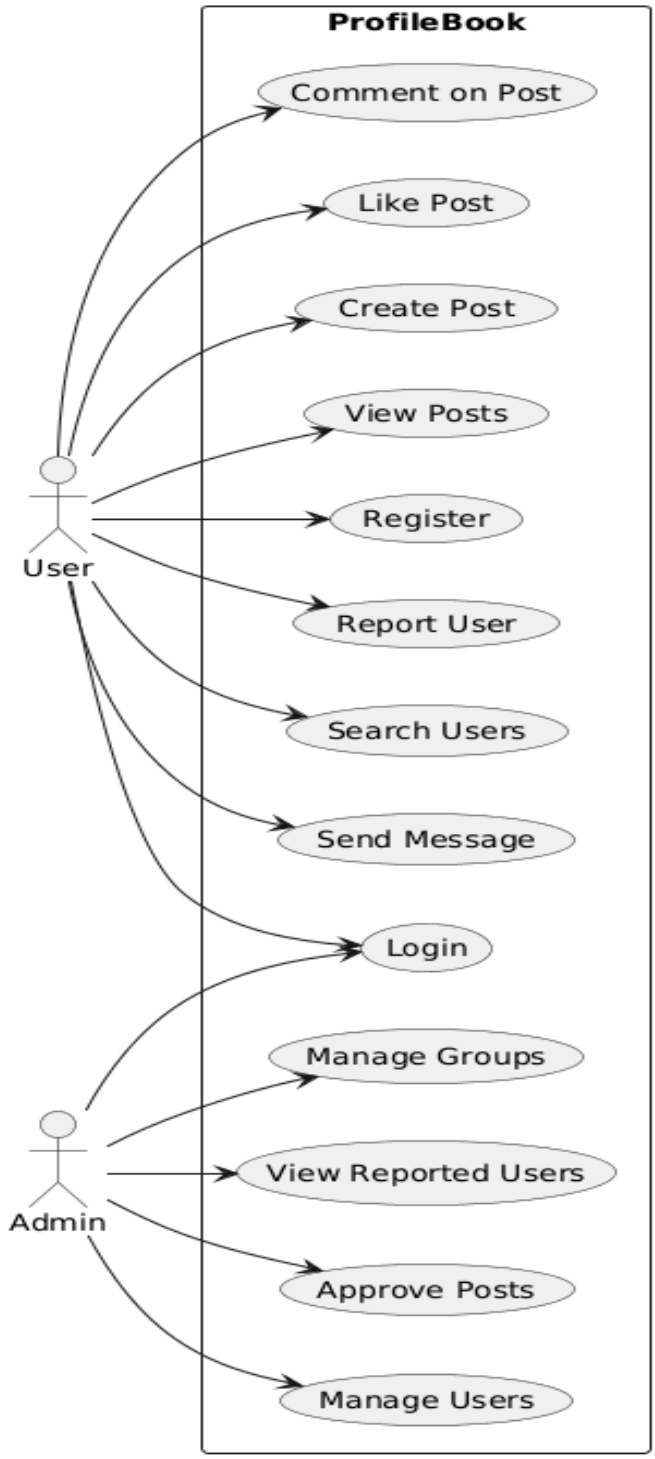
Login

Signup

Message

Report

Post


ProfileBook
Comment on Post
Like Post
Create Post
View Posts
Register
Report User
Search Users
Send Message
Login
Manage Groups
View Reported Users
Approve Posts
Manage Users
User
Admin

7). Database Design

The ProfileBook system uses **SQL Server** as the database management system for storing and managing application data. The database was implemented using the **Code-First approach of Entity Framework Core (EF Core)** in ASP.NET Core.

In the Code-First approach, database tables are automatically generated from the **C# model classes defined in the backend application**. These models represent the entities of the system and define the structure of the database tables. Each model includes properties that correspond to table columns, along with **data annotations** that define validation rules and constraints such as required fields, string length limits, and data formats.

Using this approach helped simplify database management and allowed the schema to evolve during development through **Entity Framework migrations**.

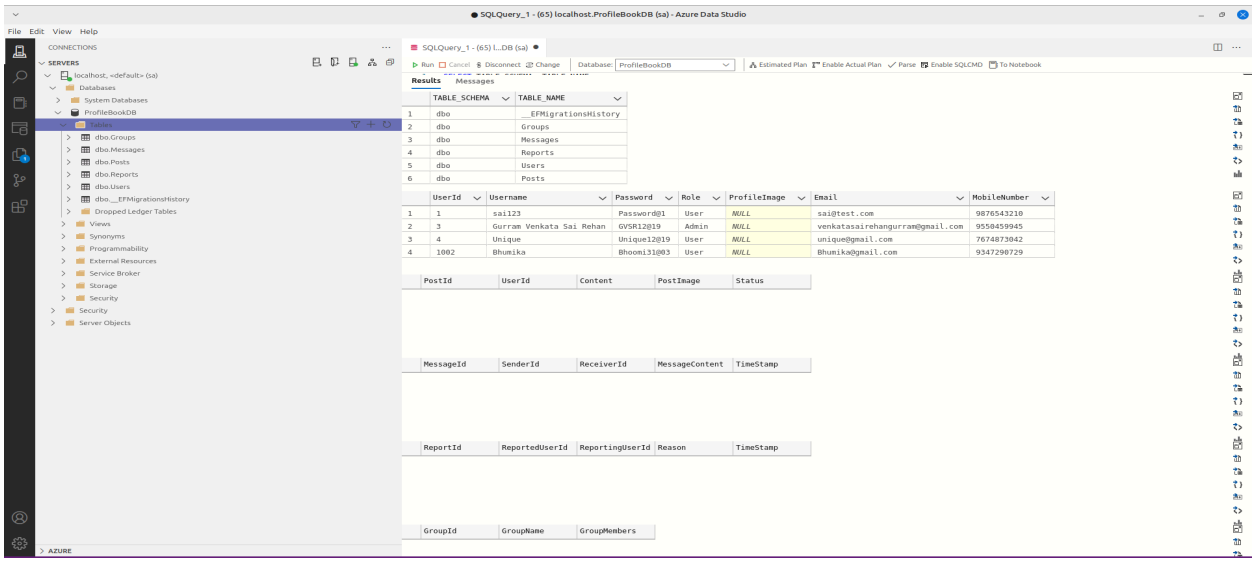
The ProfileBook database consists of the following main tables:

- **Users**
- **Posts**
- **Messages**
- **Reports**
- **Groups**

These tables store the core data required for the functioning of the ProfileBook platform.

Figure 7.1: ProfileBook Database Tables

Sample DB ,Tables,Pic this is taken after done testing because I thought It looks professional



7.1 Users Table

The **Users** table stores information about all registered users of the system, including both normal users and administrators.

The table includes the following fields:

- **UserId** – Unique identifier for each user (Primary Key)
- **Username** – Name of the user
- **Email** – Email address used for authentication
- **Password** – Password used for login
- **MobileNumber** – Contact number of the user
- **Role** – Specifies whether the user is a normal user or an administrator
- **ProfileImage** – Stores the reference to the user's profile image

This table acts as the central entity in the system and is referenced by other tables such as posts, messages, and reports.

The Users table is generated from the **User model class** using Entity Framework Core.

7.2 Posts Table

The **Posts** table stores posts created by users within the platform.

The table contains the following attributes:

- **PostId** – Unique identifier for each post (Primary Key)
- **UserId** – Identifier of the user who created the post
- **Content** – Text content of the post
- **PostImage** – Image associated with the post
- **Status** – Represents the approval status of the post

Each post is associated with a specific user through the **UserId**, which establishes a relationship between users and their posts.

7.3 Messages Table

The **Messages** table stores messages exchanged between users of the system.

The table contains the following attributes:

- **MessageId** – Unique identifier for each message (Primary Key)
- **SenderId** – The user who sends the message
- **ReceiverId** – The user who receives the message
- **MessageContent** – The content of the message
- **TimeStamp** – The date and time when the message was sent

This table enables communication between users within the ProfileBook platform.

7.4 Reports Table

The **Reports** table stores reports submitted by users against other users for inappropriate behavior.

The table contains the following fields:

- **ReportId** – Unique identifier for each report (Primary Key)
- **ReportedUserId** – The user who is being reported
- **ReportingUserId** – The user who submitted the report
- **Reason** – The reason provided for reporting the user
- **TimeStamp** – The time when the report was submitted

This table helps administrators monitor platform activity and take appropriate actions when violations occur.

7.5 Groups Table

The **Groups** table stores information about user groups created within the system.

The table contains the following attributes:

- **GroupId** – Unique identifier for each group (Primary Key)
- **GroupName** – Name of the group
- **GroupMembers** – Information about users belonging to the group

Groups allow administrators to organize users within the system.

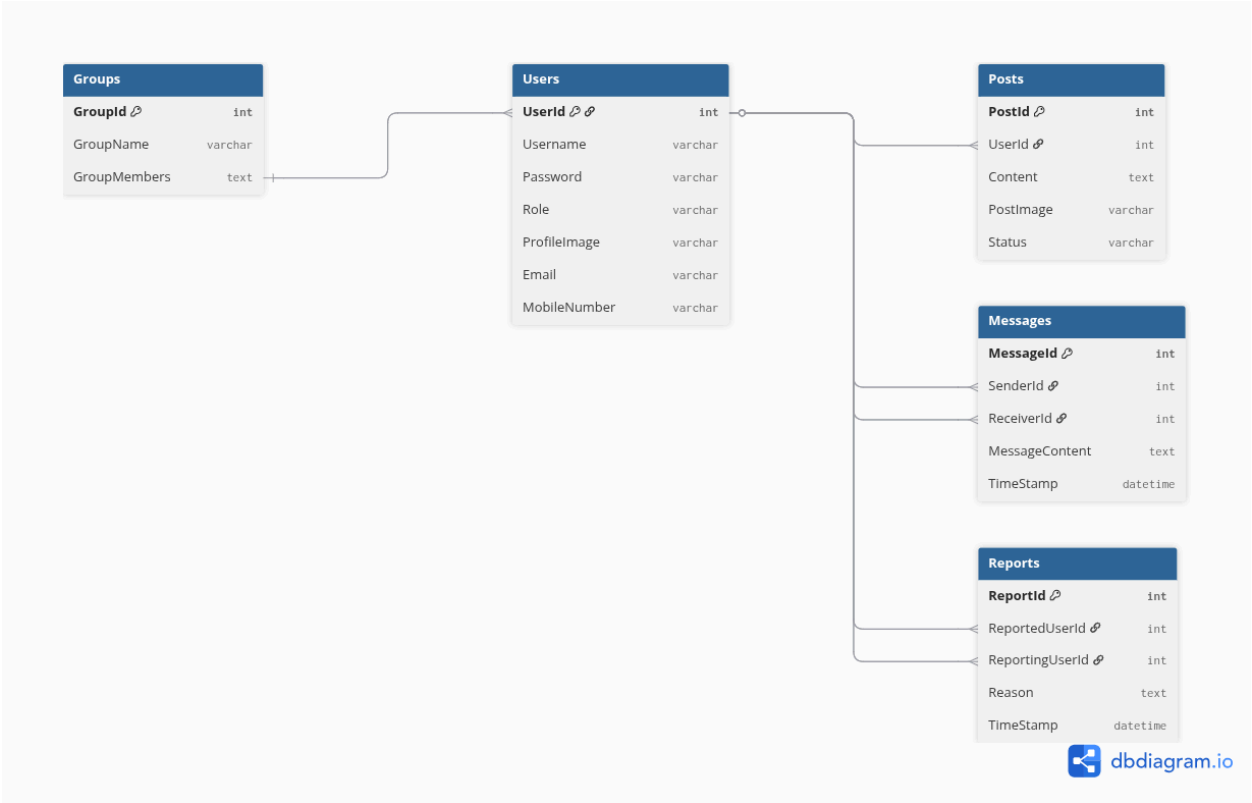
7.6 Database Implementation Using Entity Framework Core

All database tables in the ProfileBook system were created using **Entity Framework Core models with data annotations**. These annotations define validation rules and constraints directly in the model classes.

For example, the **User model** defines attributes such as required fields, string length limits, and format validation for email and mobile numbers. When migrations are executed, Entity Framework automatically generates the corresponding database schema in SQL Server.

This approach ensures that the **application models and database schema remain synchronized**, making the system easier to maintain and extend during development.

ER DIAGRAM:



I GENERATED THIS FROM dbdiagram.io because i am using linux tried so many ways but i didnt find to generate ER in my Azure Data Services because on linux SSMS is not available so I have done using ADS if u want to run just change connection string and ADS was locked on my linux so I have generated this manually .

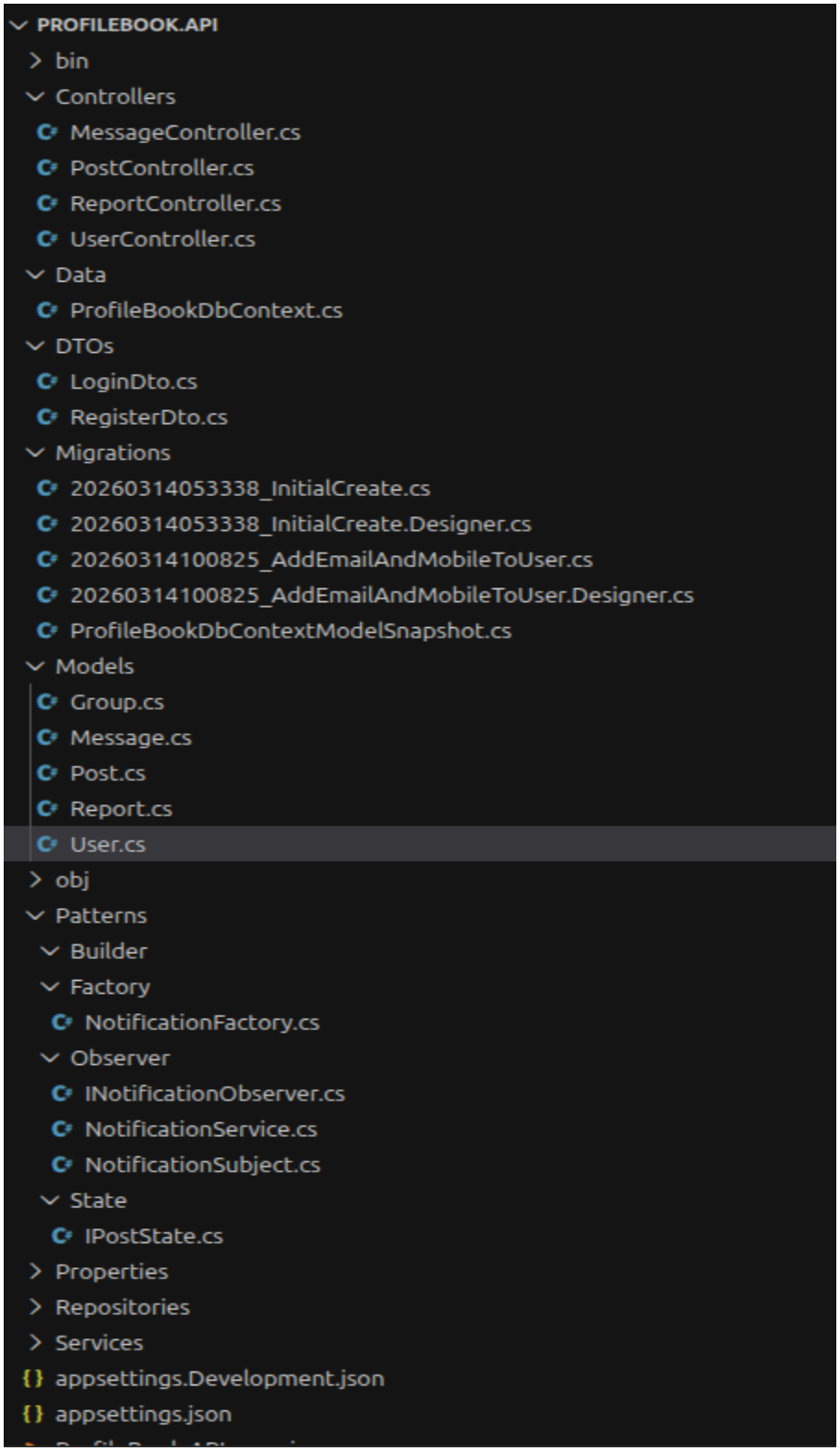
8).Backend Architecture

The backend of the ProfileBook system is developed using **ASP.NET Core Web API built on .NET 8**. The backend is responsible for handling application logic, managing database operations, and providing API endpoints that communicate with the Angular frontend.

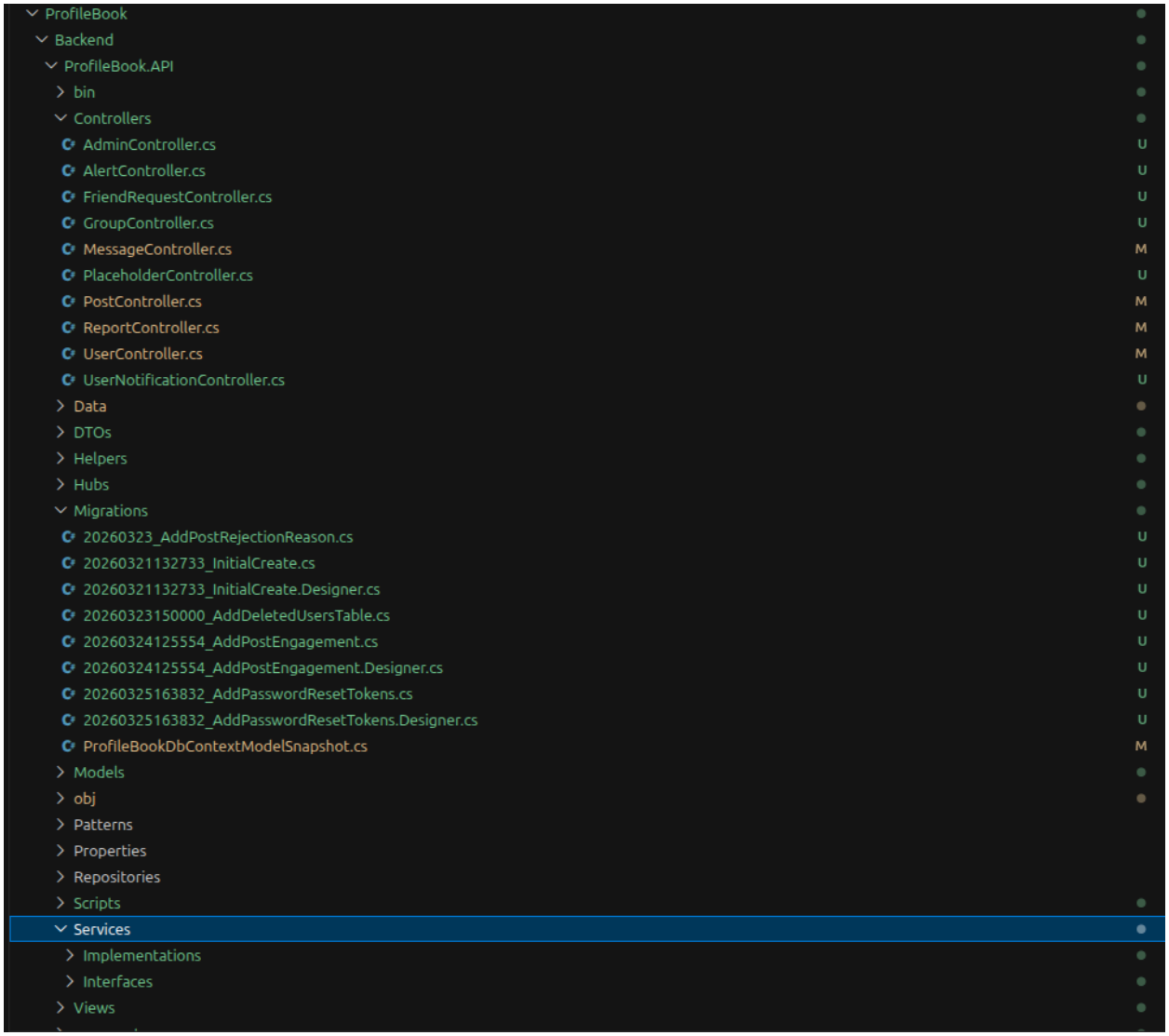
During the backend development process, I first **identified the necessary controllers and system entities** required to support the functionalities of the ProfileBook platform. Based on the system requirements, controllers were created to manage different modules such as user management, post handling, messaging, and reporting.

The backend project follows a **modular folder structure** that separates responsibilities across different components such as controllers, models, DTOs, database context, services, repositories, and design pattern implementations. This structured organization helps maintain clean architecture and improves the maintainability of the system.

Figure 8.1: Backend Project Structure



Updated Structure after Sprint -1:



8.1 Controllers

The following controllers are implemented in the final backend:

- UserController - handles registration, login, forgot password, reset password, profile retrieval, profile update, and user listing.
- AdminController - handles admin dashboard operations, user management, admin creation, credential extension, and admin profile functions.
- PostController - handles post creation, feed retrieval, likes, comments, sharing, and post moderation.
- MessageController - handles direct messaging, group messaging, and message read updates.
- ReportController - handles reporting of users and administrative action on reports.
- GroupController - handles group creation, group listing, and deletion.
- FriendRequestController - handles sending, responding to, and viewing friend requests.
- AlertController - handles admin broadcast alerts and alert delivery.
- UserNotificationController - handles retrieval and read-status updates for user notifications.
- PlaceholderController - provides MVC placeholder pages included for architectural completeness.

8.2 Models

The backend models represent the database entities of the system. Core models include User, Post, Message, Report, and Group. Additional models such as FriendRequest, UserNotification, AlertMessage, PostLike, PostComment, PostShare, DeletedUser, and PasswordResetToken extend the schema to support the final implemented features.

8.3 Data (DbContext)

The `ProfileBookDbContext` class is responsible for managing the connection between the application and the SQL Server database. It defines the `DbSet` collections for all entities and configures unique indexes such as email, username, mobile number, post-like uniqueness, and password reset token uniqueness.

8.4 DTOs

DTOs are used extensively in the backend to safely transfer data between the frontend and backend without directly exposing entity models. In the final implementation, DTOs are used for login, registration, forgot password, reset password, post creation, post commenting, post sharing, admin creation, user update, group creation, report creation, report action, messaging, alert broadcasting, and friend request processing.

This DTO-based approach improves validation, reduces tight coupling, and ensures that only the required data is accepted by each API endpoint.

8.5 Migrations

The Migrations folder contains Entity Framework Core migration files that track changes in the database schema across development stages. These migrations include the initial schema, deleted users table, post rejection reason support, post engagement tables, and password reset token support.

8.6 Design Patterns

The backend contains design pattern implementations and pattern-oriented helpers:

- Factory Pattern for notification creation
- Observer Pattern for notification/event propagation
- State Pattern for post state representation such as Pending, Approved, and Rejected

These patterns improve organization and make the code easier to extend.

8.7 Services and Repositories

The backend includes a service layer that separates business logic from controller logic. The main service classes implemented are `UserService` and `AdminService`. These services are responsible for operations such as authentication, registration validation, password reset handling, admin creation, user updates, admin creough the original report mentioned a repository layer as part of the architecture, the final code relies primarily on Entity Framework Core `DbContext` and the service layer for data access and business coordination. This still provides structured separation of responsibilities and keeps the backend maintainable.

8.8 Helpers and Authentication

The backend includes helper classes to centralize reusable logic. `AuthHelper` is used to resolve the current user identity from JWT claims and bearer tokens. `PasswordHelper` manages password hashing and verification, while `PasswordPolicyHelper` enforces password rules such as uppercase, special character, and minimum length

requirements. `IdentityNormalizer` is used to normalize email addresses, usernames, mobile numbers, and login identifiers for consistent comparison and validation.

JWT authentication is implemented to protect user and admin routes. Role-based authorization is used to restrict access to admin-only operations, and additional main-admin checks are used for highly privileged actions.

8.9 SignalR Real-Time Communication

SignalR is used in the backend to provide real-time communication capabilities. A `ChatHub` was implemented to support direct messaging, group messaging, typing indicators, message read notifications, and live application notifications.

This real-time layer improves user interaction by allowing messages and notifications to appear immediately without requiring full page reloads. It is also used to notify admins about newly submitted posts and to push emergency alerts and user notifications live to connected clients.dential governance, and user deletion workflows.

Although the original report mentioned a repository layer as part of the architecture, the final code relies primarily on Entity Framework Core `DbContext` and the service layer for data access and business coordination. This still provides structured separation of responsibilities and keeps the backend maintainable.

8.8 Helpers and Authentication

The backend includes helper classes to centralize reusable logic. `AuthHelper` is used to resolve the current user identity from JWT claims and bearer tokens. `PasswordHelper` manages password hashing and verification, while `PasswordPolicyHelper` enforces password rules such as uppercase, special character, and minimum length requirements. `IdentityNormalizer` is used to normalize email addresses, usernames, mobile numbers, and login identifiers for consistent comparison and validation.

JWT authentication is implemented to protect user and admin routes. Role-based authorization is used to restrict access to admin-only operations, and additional main-admin checks are used for highly privileged actions.

8.9 SignalR Real-Time Communication

SignalR is used in the backend to provide real-time communication capabilities. A `ChatHub` was implemented to support direct messaging, group messaging, typing indicators, message read notifications, and live application notifications.

This real-time layer improves user interaction by allowing messages and notifications to appear immediately without requiring full page reloads. It is also used to notify admins about newly submitted posts and to push emergency alerts and user notifications live to connected clients.

9. Frontend Implementation

The frontend of the ProfileBook system is implemented using Angular with standalone components, routing, route guards, service-based API communication, and SignalR integration for live functionality. The frontend is no longer limited to a basic static dashboard, but now includes a complete set of user and admin modules connected directly to the backend APIs.

The final frontend includes the following implemented pages:

- Welcome Page
- Login Page
- Signup Page
- Forgot Password Page
- Reset Password Page
- User Dashboard
- User Messages Page
- User Notifications Page
- User Friend Requests Page
- User Search Users Page
- User Profile Page
- Admin Dashboard
- Manage Users Page
- Reports Page
- Review Posts Page
- Groups Page

These pages interact with the backend using a shared fetch-based API client service. Authentication state, user state, and admin state are handled using Angular services and route guards.

9.1 Public Pages

The public module includes the Welcome page, Login page, Signup page, Forgot Password page, and Reset Password page. These pages provide the entry points for authentication and account recovery.

9.2 User Module

The user module includes the User Dashboard, User Messages page, User Notifications page, User Friend Requests page, User Search Users page, and User Profile page.

The user dashboard supports post creation, post feed viewing, likes, comments, sharing, alerts, and notification handling. The messages page supports direct messaging, group messaging, unread tracking, typing indicators, and read receipts. The profile page allows username updates and profile image uploads. The search and friend-request pages support social connectivity features.

9.3 Admin Module

The admin module includes the Admin Dashboard, Manage Users page, Reports page, Review Posts page, and Groups page.

The admin dashboard shows system statistics and expiring admin credential information. The Manage Users page supports user creation, admin creation, updates, and deletions. The Reports page supports moderation decisions. The Review Posts page

supports post approval, rejection, and deletion. The Groups page supports group creation and deletion.

9.4 State Management and API Handling

The frontend uses a shared fetch-based API client service to communicate with the ASP.NET Core backend. Authentication tokens are stored in session storage and automatically included in authorized requests.

To manage shared UI data, reducer-style Angular services were implemented for both user and admin modules. These services allow components to reactively share information such as dashboard statistics, posts, notifications, groups, and user management data. Although React Context API is not applicable in Angular, the implemented reducer-style services satisfy the same requirement intent of centralized state handling.

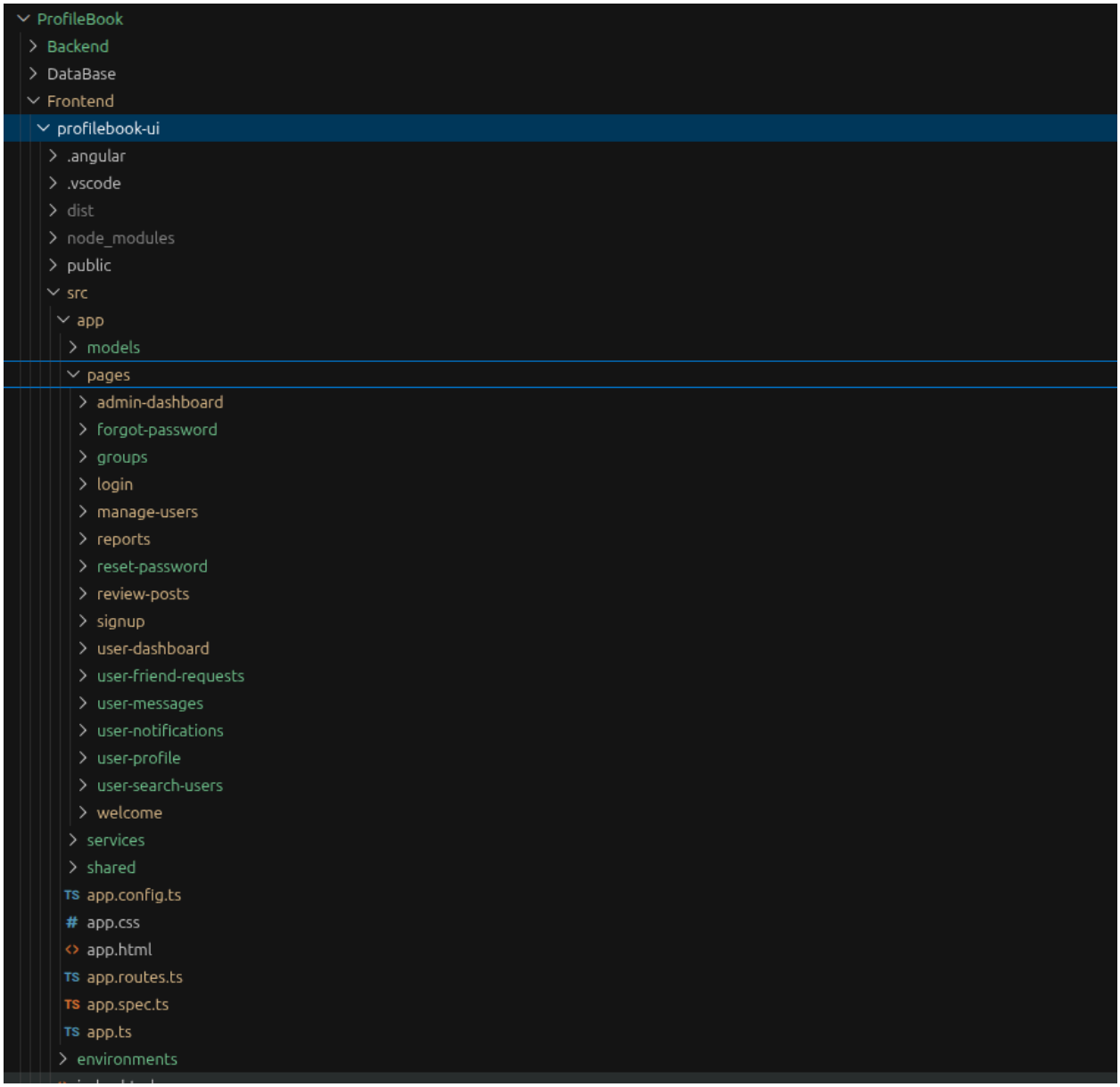
9.5 Route Guards and Authentication Flow

The frontend includes route guards to protect user and admin pages. `UserGuard` restricts user routes to authenticated users with the User role. `AdminGuard` restricts admin routes to authenticated administrators. `MainAdminGuard` provides an additional layer of protection for features that should only be accessible to the Main Admin, such as sensitive user management and group administration.

This route protection ensures that the Angular frontend aligns with the backend authorization model based on JWT roles and custom main-admin claims.

Above frontend components interact with the backend APIs developed using **ASP.NET Core Web API (.NET 8)** to perform operations such as user authentication and data retrieval.

Frontend Structure:



10. Testing

Testing was performed across backend APIs, frontend pages, and integrated workflows. Swagger was used to verify backend endpoints, and a Postman collection is available in the repository for API testing. Frontend behavior was tested through functional navigation, form validation, login flows, and dashboard behavior.

The available testing screenshots in the repository cover welcome page verification, successful registration, static user login, admin login, admin dashboard behavior, Swagger smoke testing, fetch post testing, signup testing, and login testing. These screenshots provide direct visual evidence of system execution and support the capstone demonstration.

In addition to smoke and functional testing, integration testing was performed to confirm that Angular forms and actions correctly communicate with the ASP.NET Core backend and SQL Server database. Real-time features such as notifications, message flow, and moderation refresh behavior were also verified during development.

10.1 Smoke Testing

Smoke testing was used to confirm that essential features such as API startup, registration, login, Swagger access, and post retrieval were functioning correctly.

10.2 Functional Testing

Functional testing was performed on user registration, login, admin login, dashboard navigation, post creation, and admin dashboard access.

10.3 Integration Testing

Integration testing verified the communication between Angular frontend components, backend APIs, SignalR, and SQL Server persistence.

10.4 API Testing

Swagger and Postman were used to validate request and response behavior of the implemented API endpoints.

10.5 Real-Time Verification

Real-time verification covered message delivery, typing indicators, message read handling, app notifications, and admin alert delivery.

All screenshots and materials related to the testing process are available in the **GitHub repository of the project**. The testing images can be found inside the **"Testing" folder** in the repository.

GitHub Repository Link:

https://github.com/GVSRehan/ProfileBook_Wipro_Training_Venkata_Sai_Rehan

11. Final Completion Status

The earlier version of this report described several major functionalities as upcoming work under Sprint 2 and Sprint 3. After comparing that report with the current repository, those planned deliverables are now implemented.

Completed items include:

- Angular frontend with routing and multiple modules
- User authentication and admin authentication
- JWT-based authorization
- User CRUD-related admin management
- Profile management with image upload
- Post creation, approval, rejection, likes, comments, and sharing
- User search and friend request flow
- Direct and group messaging
- User reporting and admin report actions
- Group management
- Swagger API documentation
- SignalR real-time communication
- User notifications and emergency alert broadcasting
- Forgot password and reset password flow

Therefore, the project can now be presented as a completed full-stack capstone system rather than a partial sprint-stage implementation.

12. Future Improvements

Although the core capstone requirements and many advanced features have already been implemented, for the Production we need to implement load balancers and fingerprint authentication for more security.

13. Conclusion

ProfileBook has evolved from an initially planned sprint-based capstone project into a functionally complete social networking web application. The final system now includes user authentication, admin moderation, post workflows, social interaction features, messaging, reporting, notifications, profile management, group management, and real-time communication through SignalR.

The frontend, backend, and database are fully integrated, and the project includes both the originally required capstone features and several additional enhancements such as friend requests, emergency alerts, typing indicators, read receipts, and password reset flow.

From a capstone submission perspective, the project should now be documented and presented as a completed full-stack application. The main work remaining is not system development, but updating the report so that it accurately reflects the final implemented state of the repository.

14. Output

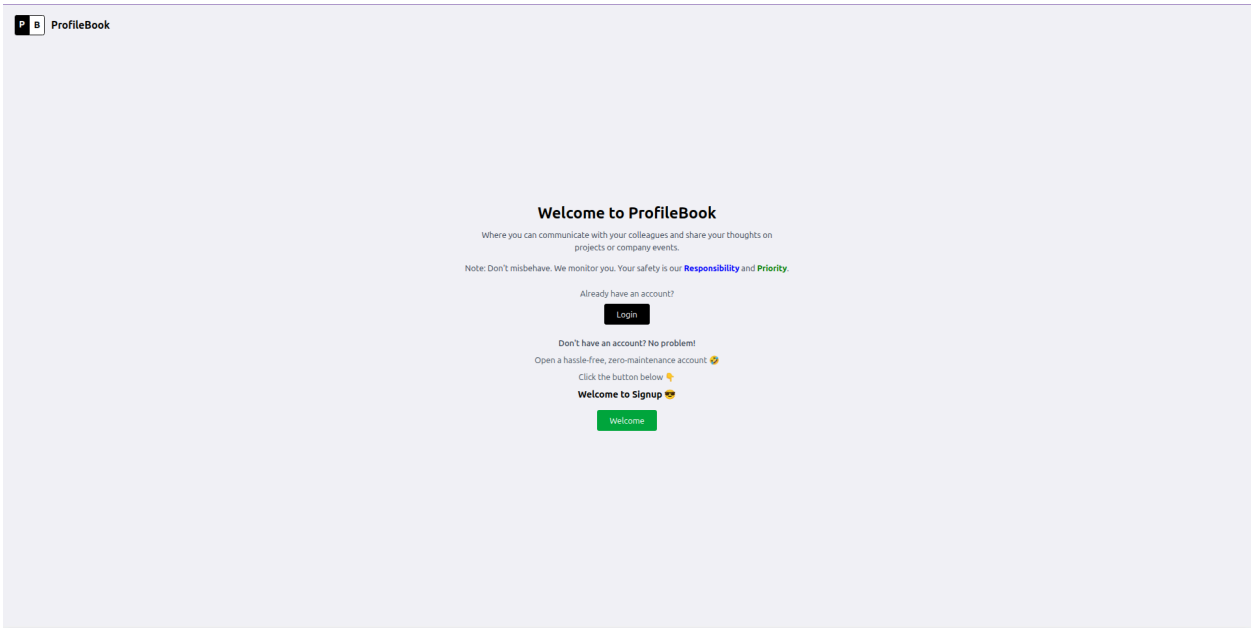
To make the project easier to evaluate without running the application, output screenshots are included in this report. These screenshots visually demonstrate the implemented frontend pages, backend API verification, and system workflows. This helps the reader understand the actual execution of the application directly from the report.

The following screenshots can be added in this section:

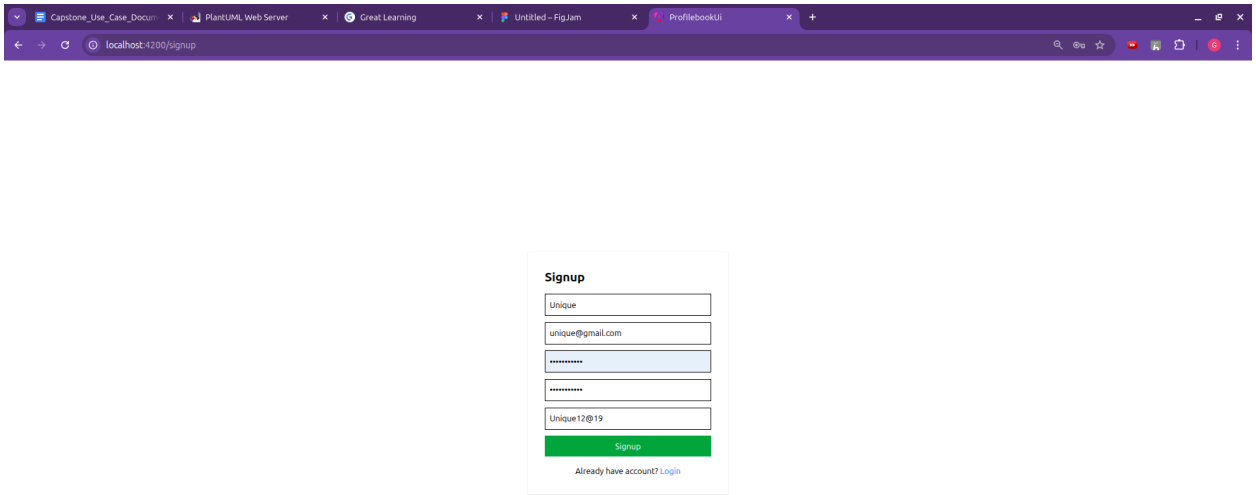
- Welcome Page
- Successful User Registration
- User Login
- Admin Login
- Admin Dashboard
- Swagger Default Page
- Signup API Testing
- Login API Testing
- Fetch Post Testing
- Post Creation Testing

GITHUB:https://github.com/GVSRehan/ProfileBook_Wipro_Training_Venkata_Sai_Rehan
Figma Brain Stroming:
https://www.figma.com/board/Ep58i50BZDZxA65sFx39Cr/Untitled?node-id=0-1&p=f&t=zmUenVerygir4lEz-0

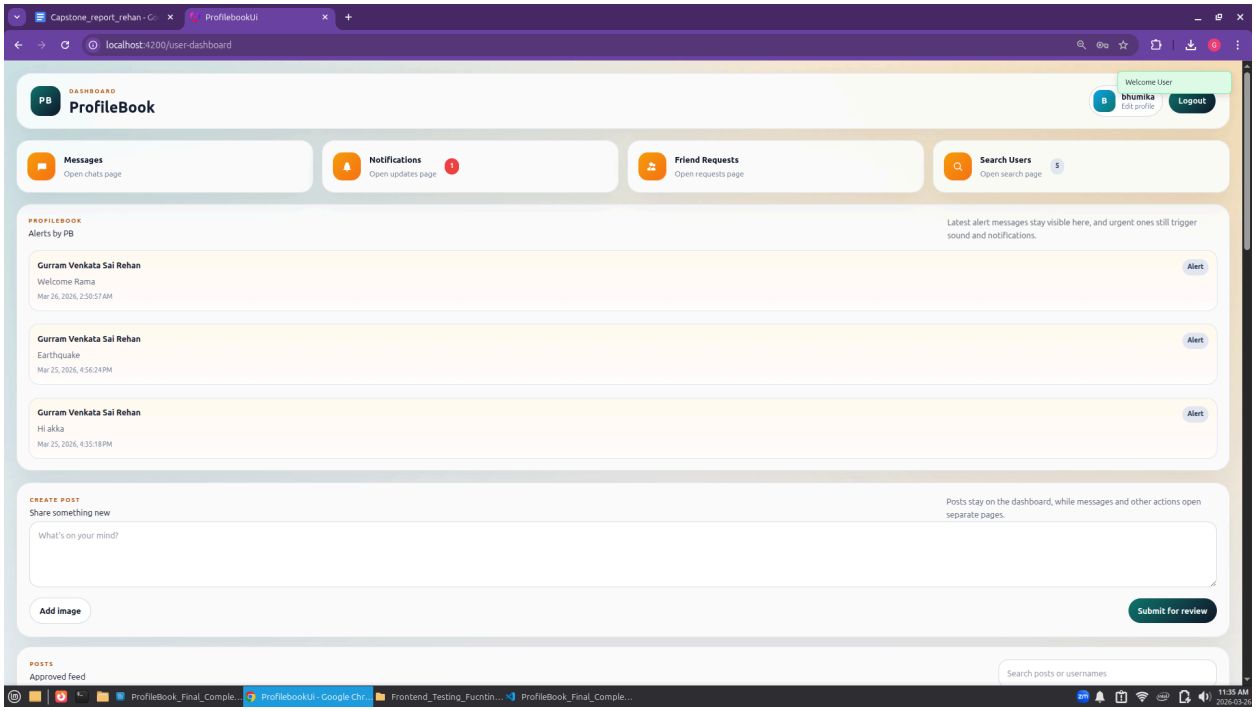
Welcome Page:



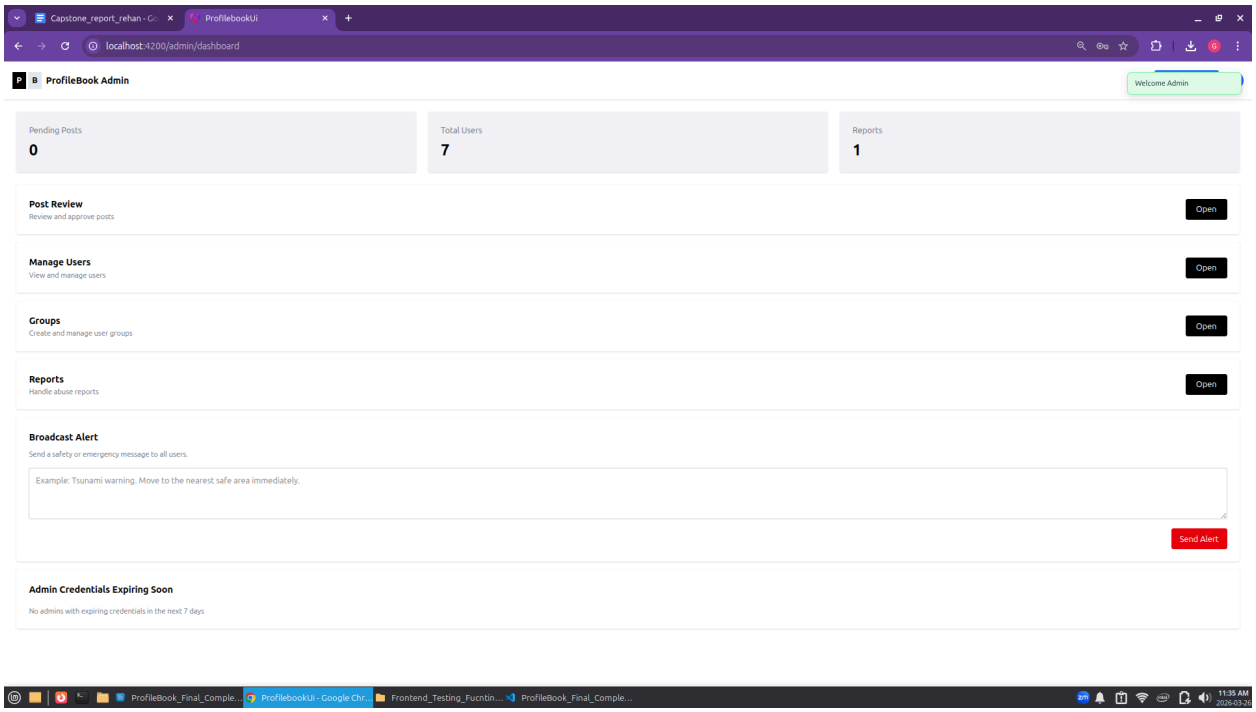
Successful User Registration:



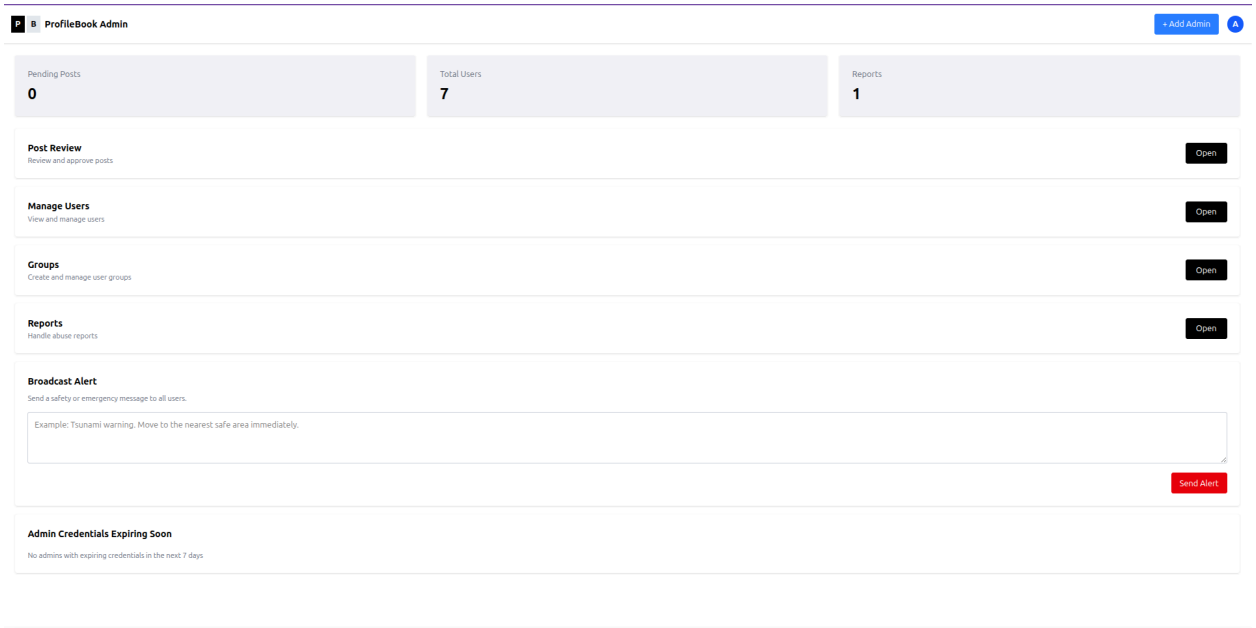
User Login DashBoard:



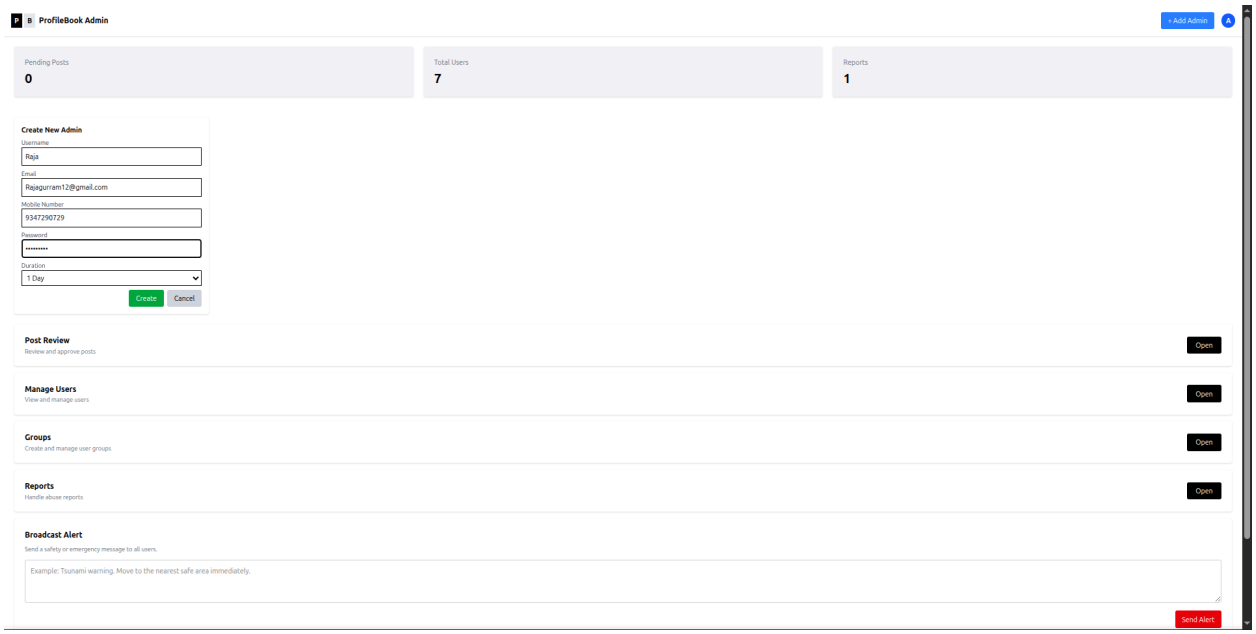
Admin Login Dashboard:



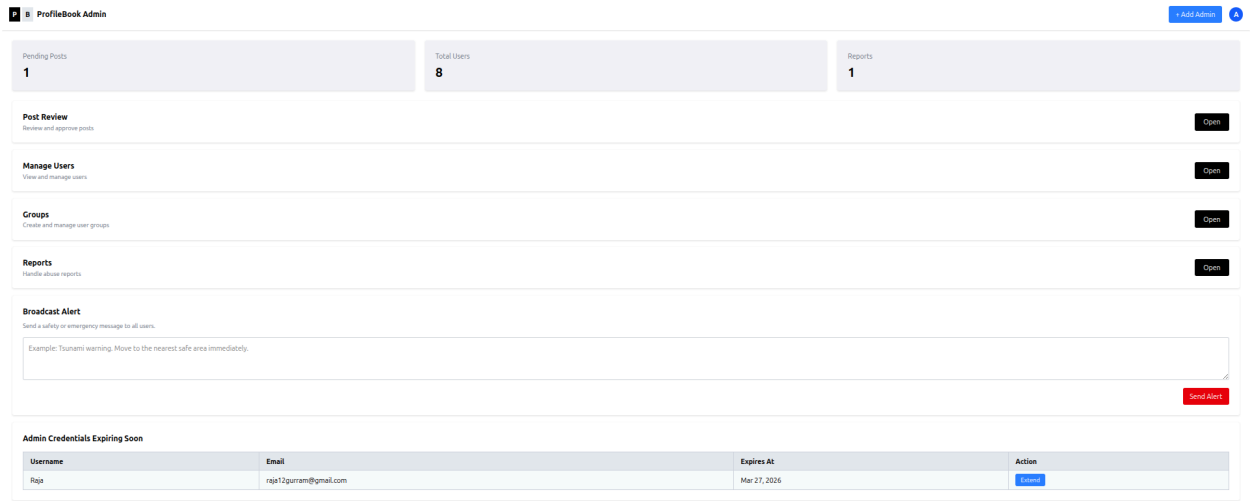
Admin Dashboard:



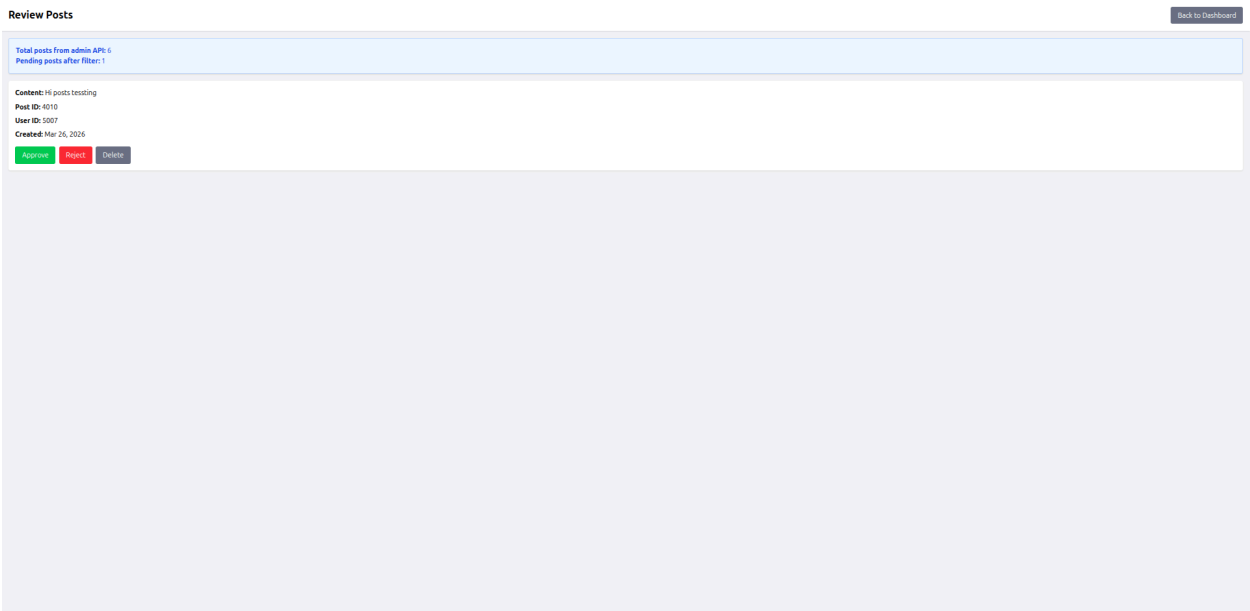
Sub_Admin_Creation:



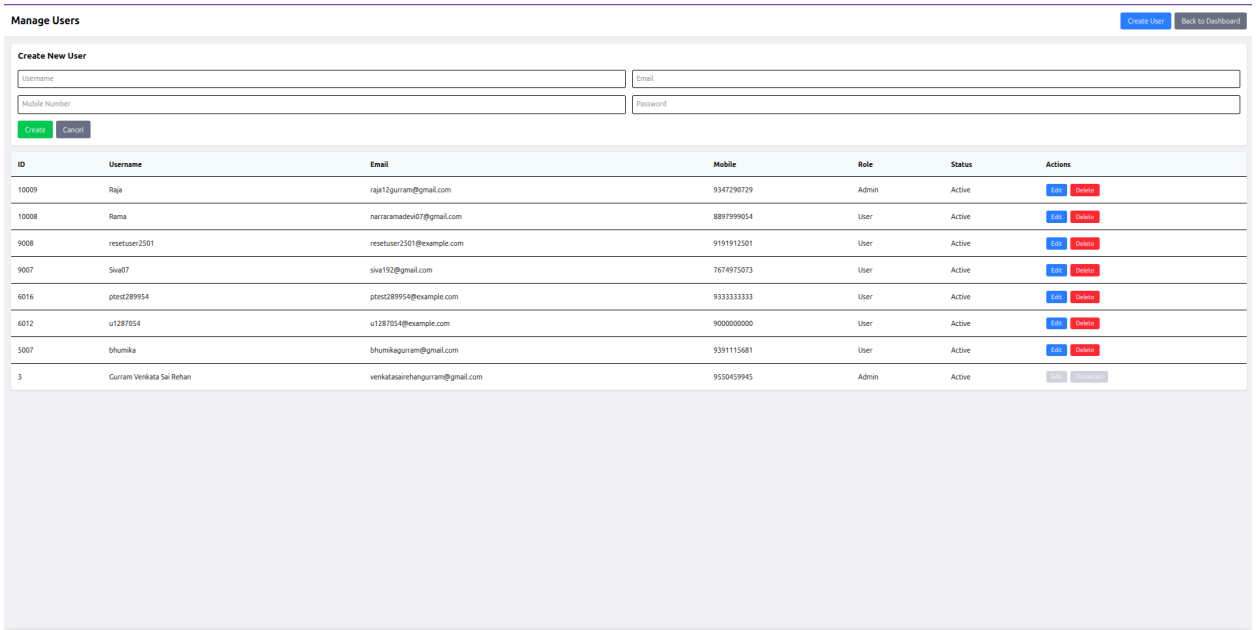
After Sub Admin Creation:



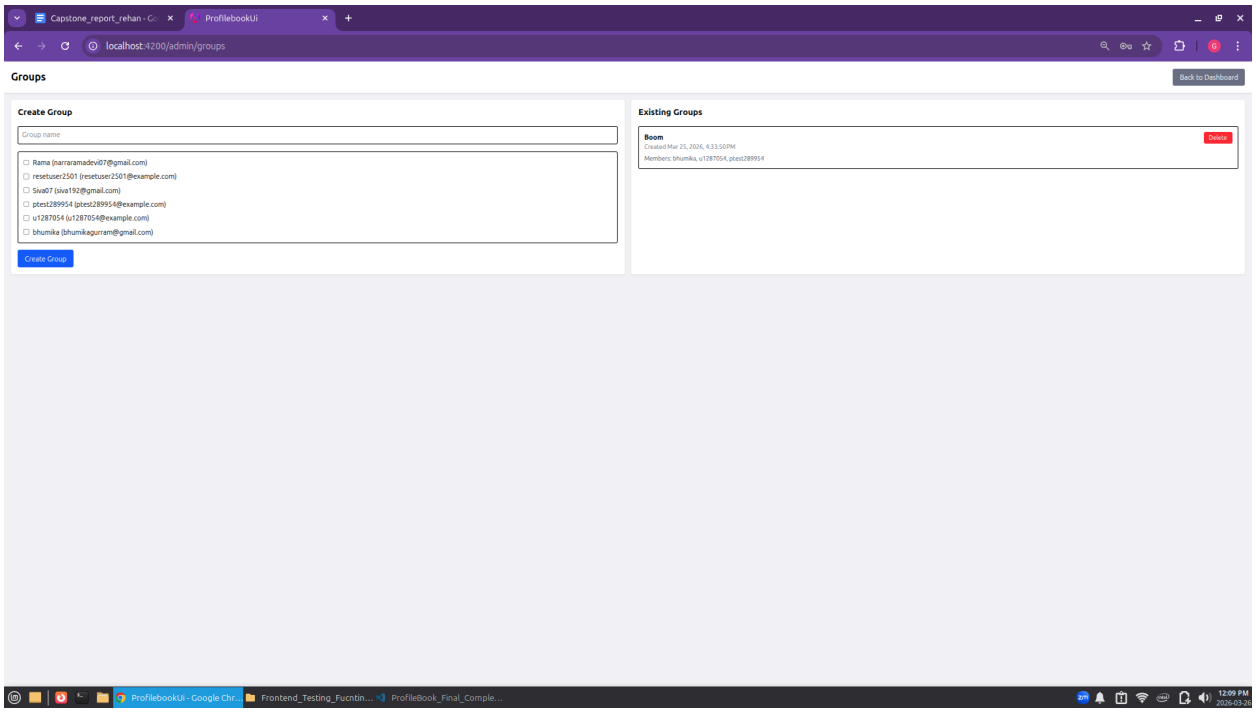
Post Review Admin side:



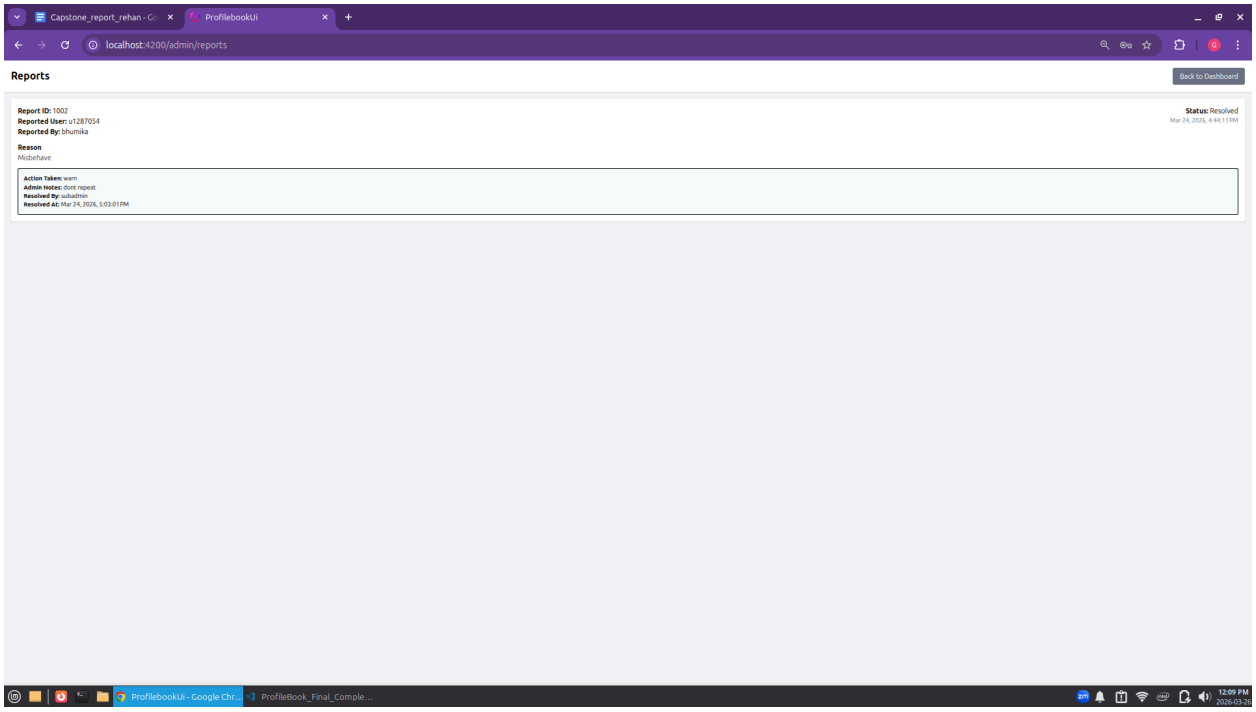
Manage Users Admin:



Group Page Admin:

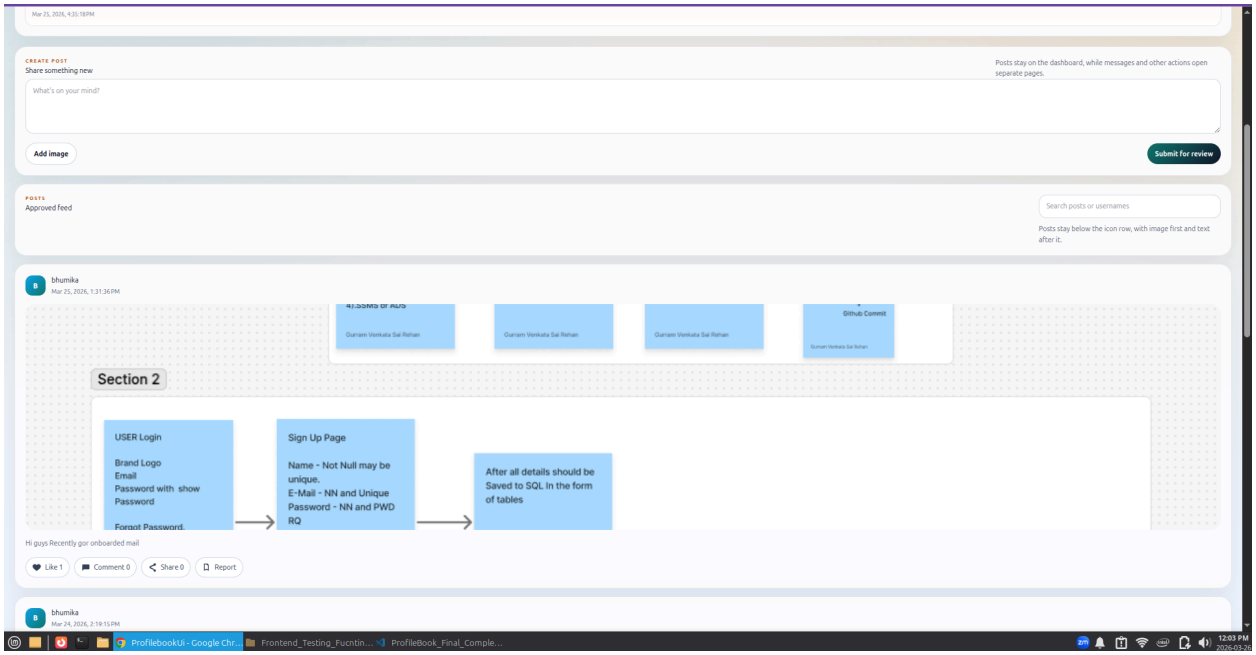


Report Page Admin:

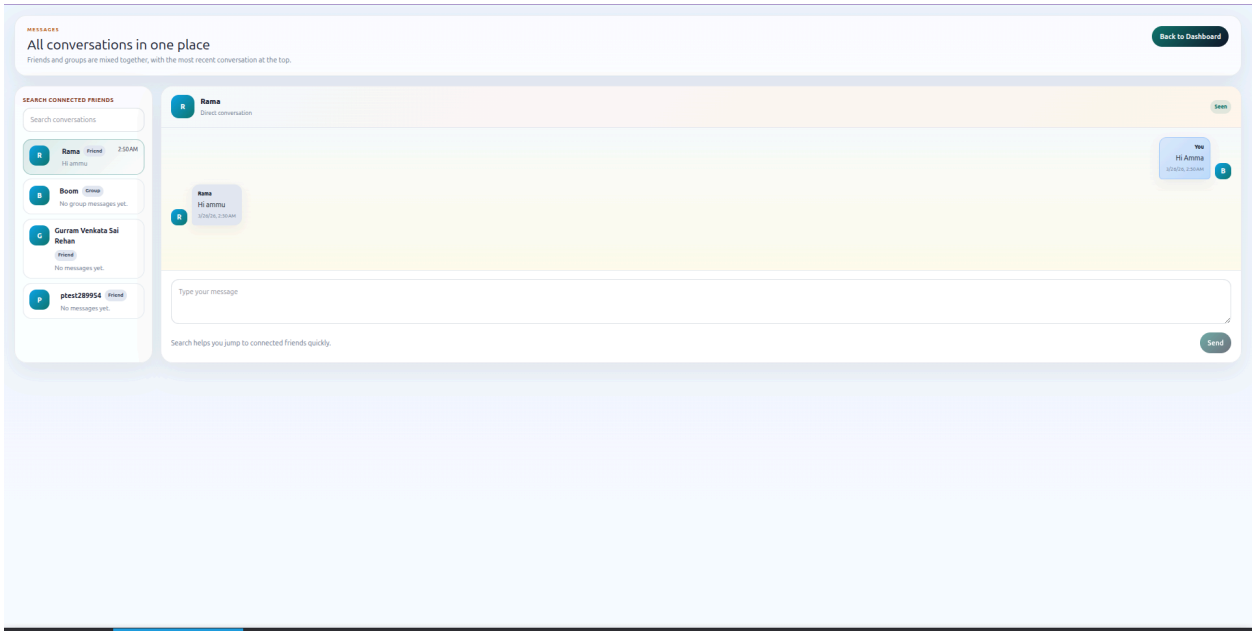


Sub Admin Dashboard:

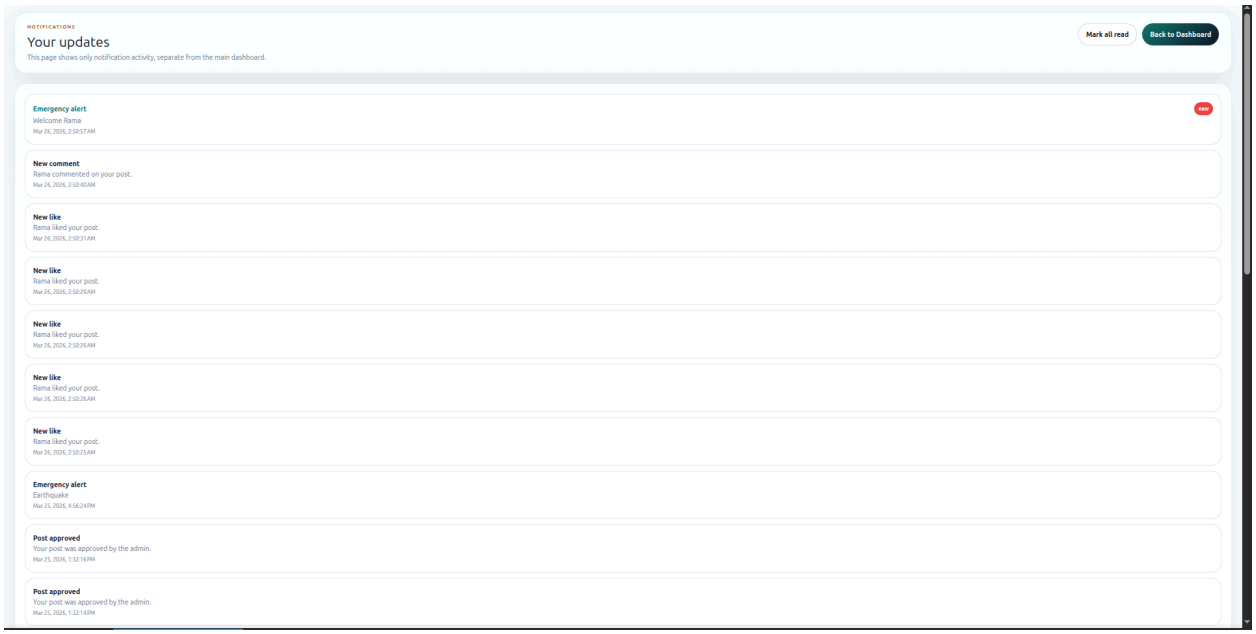
User Post Page:



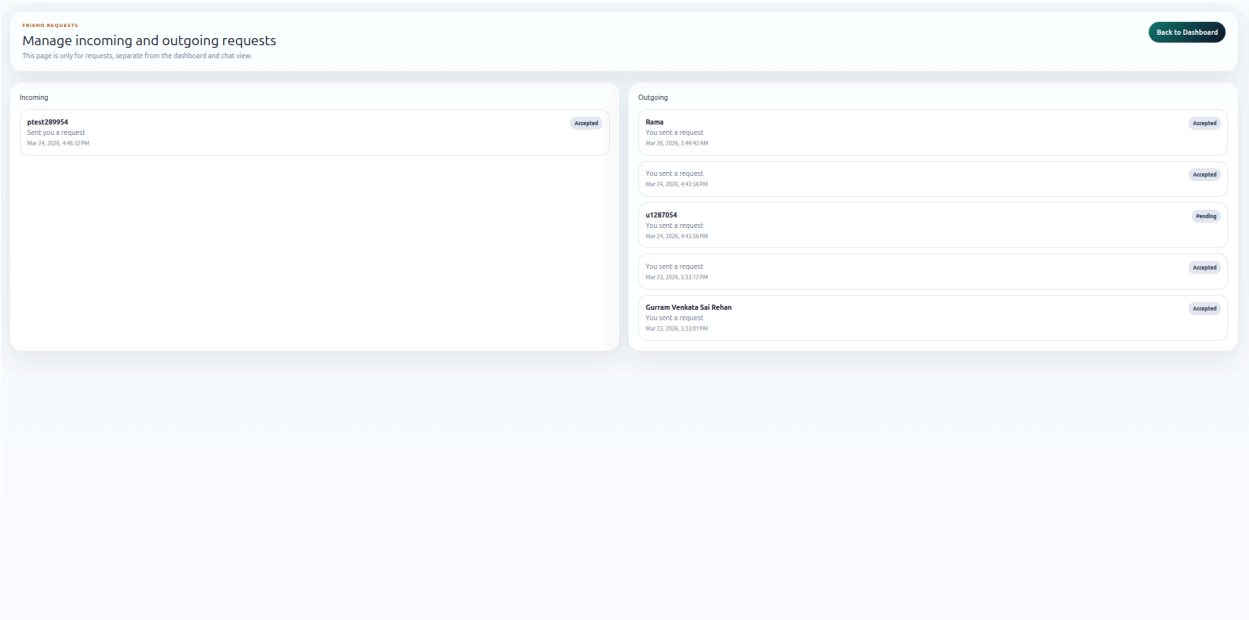
User Message Page:



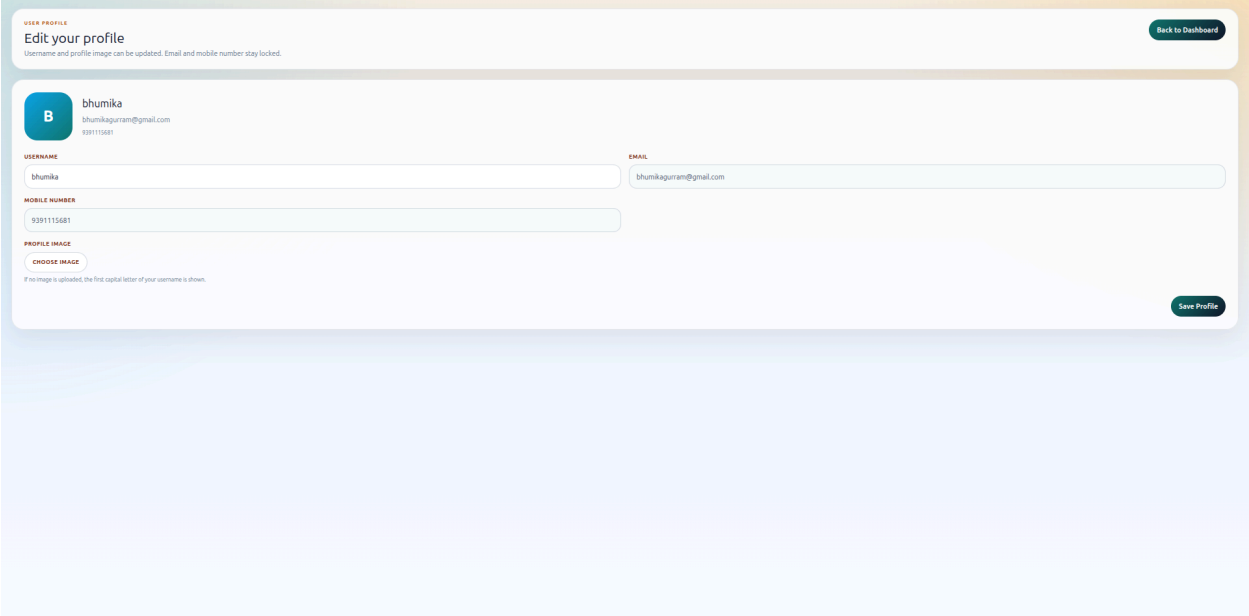
User Notification Page:



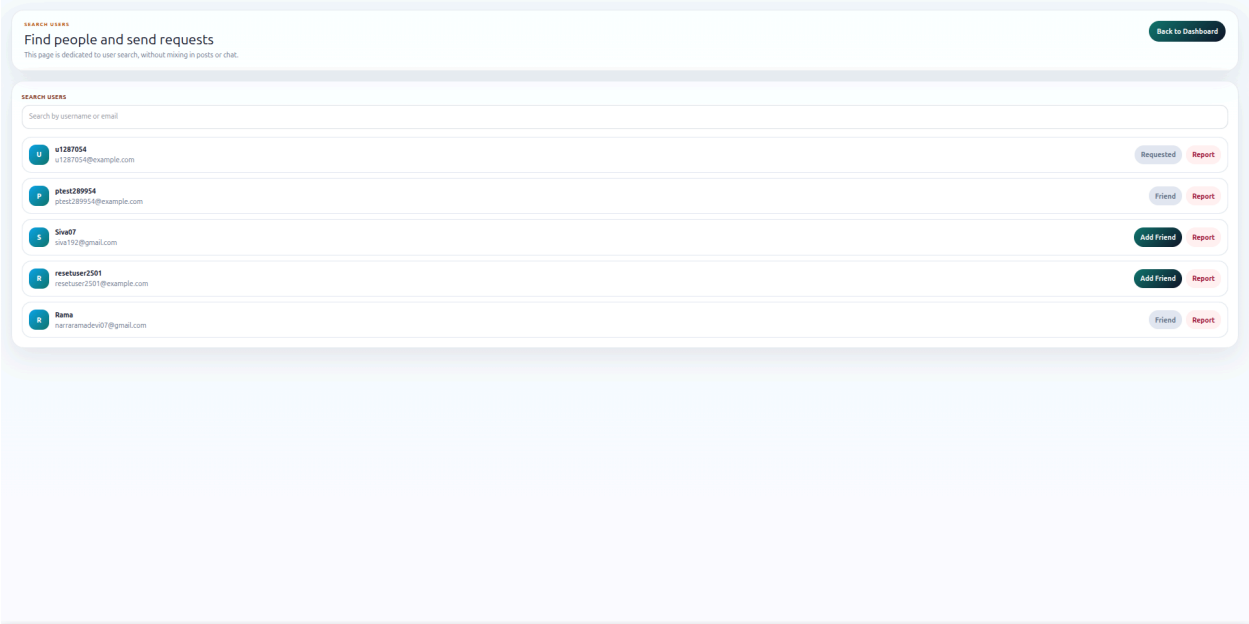
User Friend Request Page:



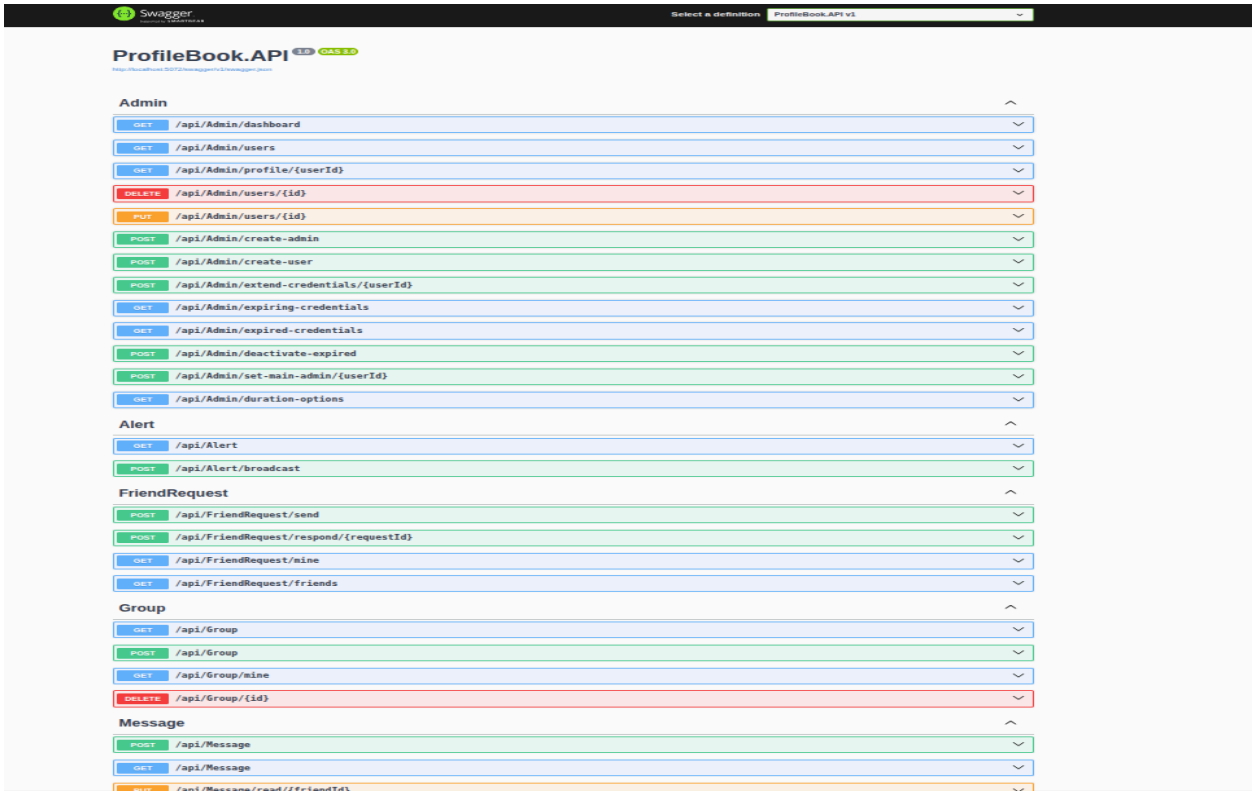
User Profile Page:



Search User Page:



Swagger Default Page



Placeholder

- GET /placeholders
- GET /placeholders/login
- GET /placeholders/register
- GET /placeholders/posts
- GET /placeholders/messages

Post

- POST /api/Post
- GET /api/Post
- POST /api/Post/{id}/like
- POST /api/Post/{id}/comments
- POST /api/Post/{id}/share
- GET /api/Post/all
- GET /api/Post/pending
- PUT /api/Post/approve/{id}
- PUT /api/Post/reject/{id}
- DELETE /api/Post/{id}

Report

- POST /api/Report
- GET /api/Report
- PUT /api/Report/{reportId}/action

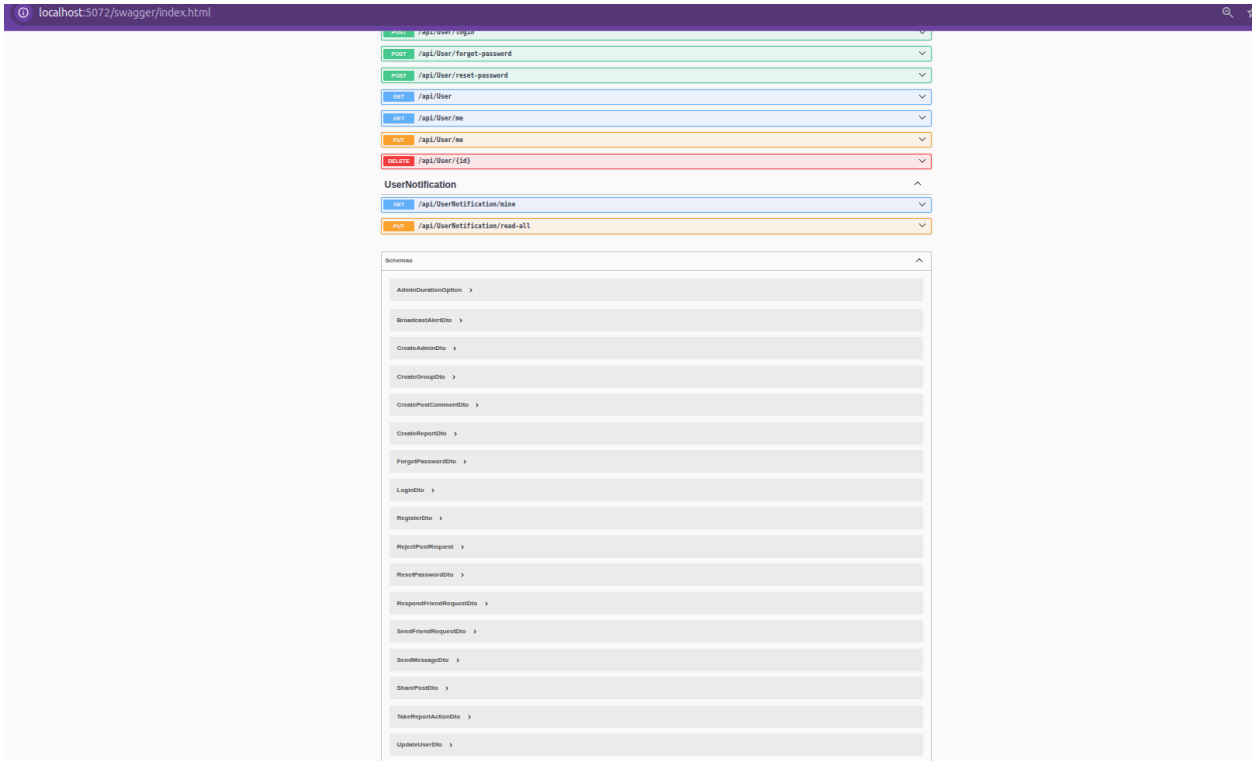
User

- POST /api/User/register
- POST /api/User/login
- POST /api/User/forgot-password
- POST /api/User/reset-password
- GET /api/User
- GET /api/User/me
- PUT /api/User/me
- DELETE /api/User/{id}

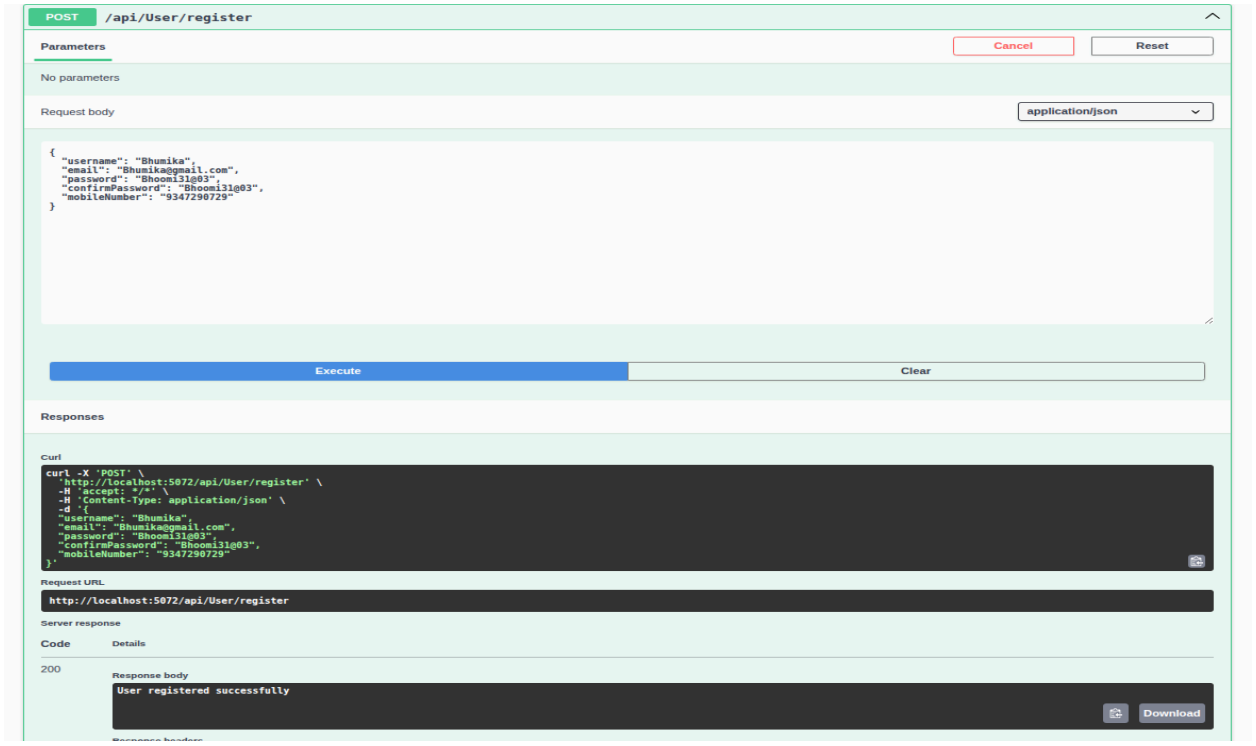
UserNotification

- GET /api/UserNotification/mine
- PUT /api/UserNotification/read-all

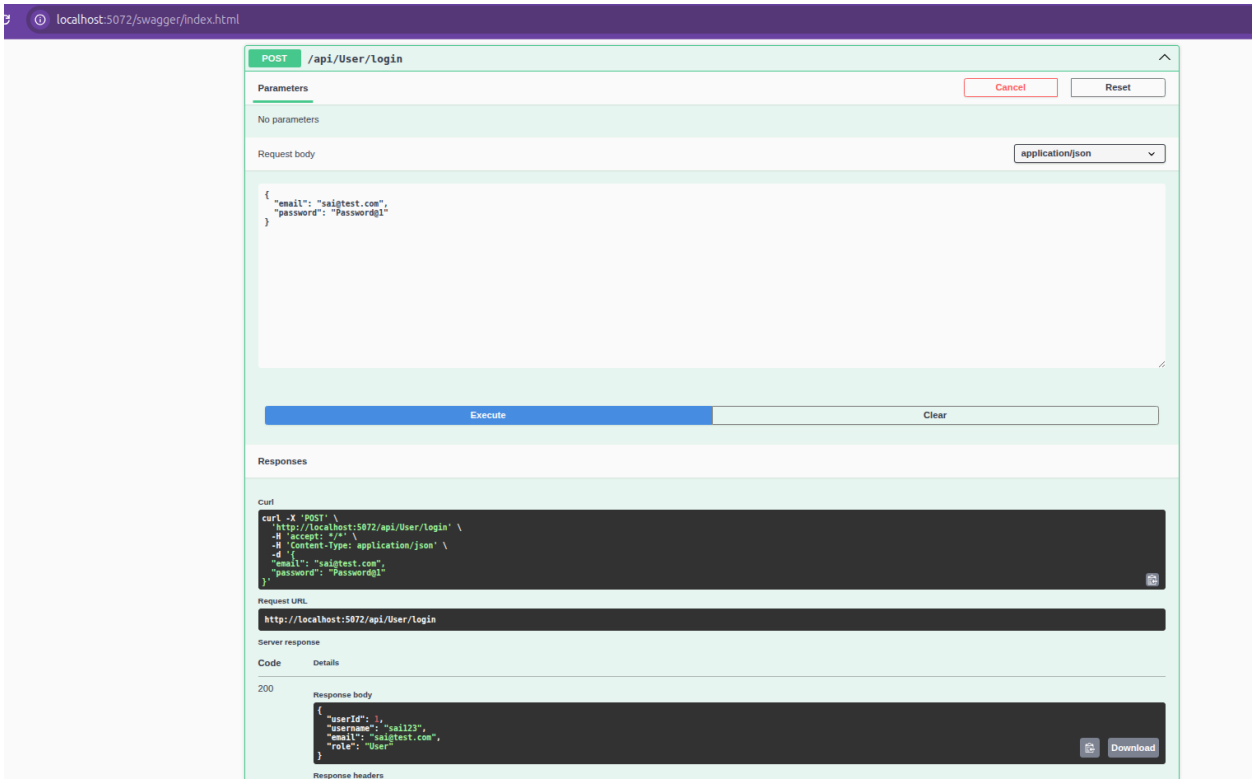
Schemas



Signup API Testing



Login API Testing



They are lots of API So All testing results provided in test folder that you can find on my github

GitHub Repository Link:

https://github.com/GVSRehan/ProfileBook_Wipro_Training_Venkata_Sai_Rehan